

notebook_lab1_extension

September 3, 2026

1 Lab 1 - MLOps Extension

1.1 Student

Joseph DELNORD

1.2 Objectives

This notebook extends the initial machine learning workflow by:

- Creating new engineered features
- Comparing Random Forest and Support Vector Machine models
- Performing hyperparameter tuning with GridSearchCV
- Selecting the best model
- Saving the model for deployment

```
[1]: import pandas as pd
import numpy as np

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import (
    train_test_split,
    GridSearchCV
)

from sklearn.pipeline import Pipeline

from sklearn.preprocessing import (
    StandardScaler,
    FunctionTransformer
)

from sklearn.ensemble import (
    RandomForestClassifier
)

from sklearn.svm import SVC
```

```

from sklearn.metrics import (
    accuracy_score,
    classification_report
)

from joblib import dump

```

```

[2]: data = load_breast_cancer(as_frame=True)

df = pd.concat(
    [data["data"], data["target"]],
    axis=1
)

print(df.shape)

df.head()

```

(569, 31)

```

[2]:
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0         17.99         10.38         122.80      1001.0         0.11840
1         20.57         17.77         132.90      1326.0         0.08474
2         19.69         21.25         130.00      1203.0         0.10960
3         11.42         20.38          77.58       386.1         0.14250
4         20.29         14.34         135.10     1297.0         0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0         0.27760         0.3001         0.14710         0.2419
1         0.07864         0.0869         0.07017         0.1812
2         0.15990         0.1974         0.12790         0.2069
3         0.28390         0.2414         0.10520         0.2597
4         0.13280         0.1980         0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0         0.07871  ...         17.33         184.60         2019.0
1         0.05667  ...         23.41         158.80         1956.0
2         0.05999  ...         25.53         152.50         1709.0
3         0.09744  ...         26.50          98.87          567.7
4         0.05883  ...         16.67         152.20         1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0         0.1622         0.6656         0.7119         0.2654
1         0.1238         0.1866         0.2416         0.1860
2         0.1444         0.4245         0.4504         0.2430
3         0.2098         0.8663         0.6869         0.2575
4         0.1374         0.2050         0.4000         0.1625

```

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

```
[3]: def add_new_features(X):

    X = X.copy()

    X["radius_area_ratio"] = (
        X["mean radius"]
        /
        (X["mean area"] + 1)
    )

    X["worst_mean_radius_diff"] = (
        X["worst radius"]
        -
        X["mean radius"]
    )

    return X
```

```
[4]: sample = add_new_features(
    df.drop(
        columns=["target"]
    )
)

sample.head()
```

```
[4]:   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0      17.99      10.38      122.80      1001.0      0.11840
1      20.57      17.77      132.90      1326.0      0.08474
2      19.69      21.25      130.00      1203.0      0.10960
3      11.42      20.38       77.58       386.1      0.14250
4      20.29      14.34      135.10      1297.0      0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0      0.27760      0.3001      0.14710      0.2419
1      0.07864      0.0869      0.07017      0.1812
2      0.15990      0.1974      0.12790      0.2069
3      0.28390      0.2414      0.10520      0.2597
```

4	0.13280	0.1980	0.10430	0.1809
---	---------	--------	---------	--------

	mean fractal dimension	...	worst perimeter	worst area	worst smoothness	\
0	0.07871	...	184.60	2019.0	0.1622	
1	0.05667	...	158.80	1956.0	0.1238	
2	0.05999	...	152.50	1709.0	0.1444	
3	0.09744	...	98.87	567.7	0.2098	
4	0.05883	...	152.20	1575.0	0.1374	

	worst compactness	worst concavity	worst concave points	worst symmetry	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	worst fractal dimension	radius_area_ratio	worst_mean_radius_diff
0	0.11890	0.017954	7.39
1	0.08902	0.015501	4.42
2	0.08758	0.016354	3.88
3	0.17300	0.029501	3.49
4	0.07678	0.015632	2.25

[5 rows x 32 columns]

```
[5]: X = df.drop(
        columns=["target"]
    )

y = df["target"]

X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.3,
        random_state=42
    )
)
```

```
[6]: preprocessing_pipeline = Pipeline([
    (
        "feature_engineering",
        FunctionTransformer(
            add_new_features
        )
    ),
])
```

```
(
    "scaler",
    StandardScaler()
)
])
```

```
[7]: models = [

    {
        "classifier": [
            RandomForestClassifier(
                random_state=42
            )
        ],

        "classifier__n_estimators": [
            50,
            100,
            200
        ],

        "classifier__max_depth": [
            None,
            10,
            20
        ]
    },

    {
        "classifier": [
            SVC()
        ],

        "classifier__C": [
            0.1,
            1,
            10
        ],

        "classifier__kernel": [
            "linear",
            "rbf"
        ]
    }

]
```

```
[8]: training_pipeline = Pipeline([

    (
        "preprocessing",
        preprocessing_pipeline
    ),

    (
        "classifier",
        RandomForestClassifier()
    )

])
```

```
[9]: grid_search = GridSearchCV(
    training_pipeline,
    models,
    cv=5,
    n_jobs=-1,
    verbose=1
)

grid_search.fit(
    X_train,
    y_train
)
```

Fitting 5 folds for each of 15 candidates, totalling 75 fits

```
[9]: GridSearchCV(cv=5,
                  estimator=Pipeline(steps=[('preprocessing',
                                             Pipeline(steps=[('feature_engineering',
                                                              FunctionTransformer(func=<function add_new_features at 0x713bbfbb4a40>)),
                                                              ('scaler',
                                                                StandardScaler()))])),
                  ('classifier',
                    RandomForestClassifier()))],
                  n_jobs=-1,
                  param_grid=[{'classifier':
[RandomForestClassifier(random_state=42)],
                              'classifier__max_depth': [None, 10, 20],
                              'classifier__n_estimators': [50, 100, 200]},
                              {'classifier': [SVC()], 'classifier__C': [0.1, 1, 10],
                              'classifier__kernel': ['linear', 'rbf']}],
                  verbose=1)
```

```
[10]: print(
        "Best Parameters:"
    )

    print(
        grid_search.best_params_
    )

    print(
        "\nBest Cross Validation Score:"
    )

    print(
        grid_search.best_score_
    )
```

Best Parameters:

```
{'classifier': SVC(), 'classifier__C': 0.1, 'classifier__kernel': 'linear'}
```

Best Cross Validation Score:

```
0.9748417721518987
```

```
[11]: best_model = (
        grid_search.best_estimator_
    )

    predictions = best_model.predict(
        X_test
    )

    print(
        "Accuracy:",
        accuracy_score(
            y_test,
            predictions
        )
    )

    print(
        classification_report(
            y_test,
            predictions
        )
    )
```

Accuracy: 0.9824561403508771

```
precision    recall  f1-score   support
```

0	0.98	0.97	0.98	63
1	0.98	0.99	0.99	108
accuracy			0.98	171
macro avg	0.98	0.98	0.98	171
weighted avg	0.98	0.98	0.98	171

```
[12]: dump(
      best_model,
      "best_cancer_model_v2.joblib"
    )

    print(
      "Model saved successfully"
    )
```

Model saved successfully

2 Conclusion

A new feature engineering strategy was implemented.

Two machine learning algorithms were compared:

- Random Forest
- Support Vector Machine

GridSearchCV was used to identify the best hyperparameters.

The best trained model was saved in:

best_cancer_model_v2.joblib

This model can now be loaded by a Flask inference pipeline for deployment.